Create the kafka topic where the log records produced:

```
docker exec -it kafka kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --create \
  --topic logs \
  --partitions 2 \
  --replication-factor 1
```

Attaching VS Code to the Spark Client container

Spark does **not** run on your host machine; it runs inside Docker containers. Attaching VS Code ensures:

- **Correct Spark version:** (4.0.0)
- **Correct Python environment**
- **Correct Kafka networking**
- **Identical setup for everyone**

> **Note:** VS Code becomes a remote UI for the `spark-client` container.

Prerequisite

Install this VS Code extension on your host:

- **Dev Containers** (Microsoft)

Attach to the running container

1. Open **VS Code**.
2. Open the **Command Palette**:
   - `Ctrl + Shift + P` (Linux/Windows)
   - `Cmd + Shift + P` (macOS)
3. Select: **Dev Containers: Attach to Running Container**.
4. Choose: **spark-client**.

VS Code will reload automatically.

Verify attachment

1. Look at the **bottom-left corner** of VS Code. It should display: `Dev Container: spark-client`
2. Open a terminal in VS Code and run:

```
spark-submit --version

OUTPUT:
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 4.0.0
      /_/

Using Scala version 2.13.16, OpenJDK 64-Bit Server VM, 17.0.15
Branch HEAD
Compiled by user wenchen on 2025-05-19T07:58:03Z
Revision fa33ea000a0bda9e5a3fa1af98e8e85b8cc5e4d4
Url https://github.com/apache/spark
Type --help for more information.
```

3. open the folder `/opt/spark-apps/`

Understanding the Spark Structured Streaming code

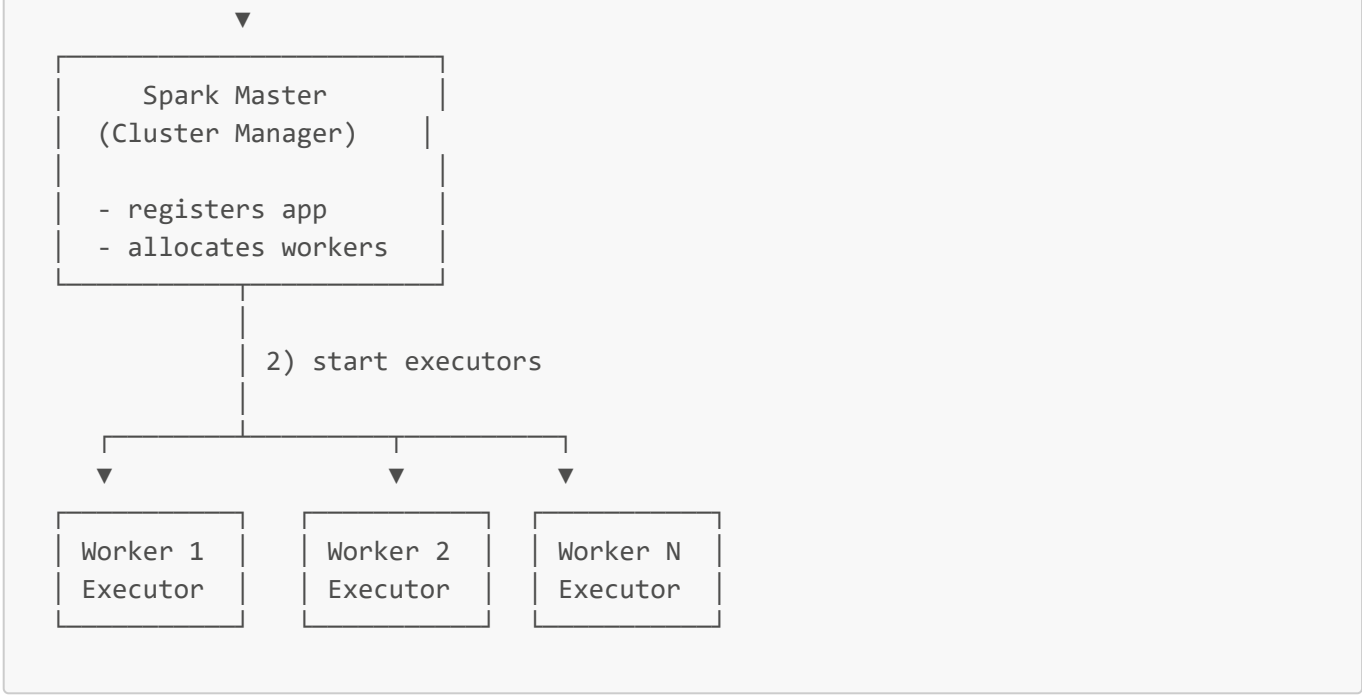
Revise the Spark Structured Streaming application example:
`spark_structured_streaming_logs_processing.py`

Running the Spark Structured Streaming application

In the spark-client terminal, example of how to run the Spark application:

```
spark-submit \
  --master spark://spark-master:7077 \
  --packages org.apache.spark:spark-sql-kafka-0-10_2.13:4.0.0 \
  --num-executors 1 \
  --executor-cores 1 \
  --executor-memory 1G \
  /opt/spark-apps/spark_structured_streaming_logs_processing.py
```

```
┌──────────────────────────┐
│    Spark Client          │
│  spark-submit            │
│  (user machine / pod)    │
└──────────┬───────────────┘
           │
           │ 1) submit application
           │
```



See the application submission in the Spark Master: http://localhost:8080 If there are no crashes, the Spark Driver should be reacheable: http://localhost:4040

Note that the python application stored locally is submitted to the spark master's URL. Also note number of executors, cores per executors, and memory management.

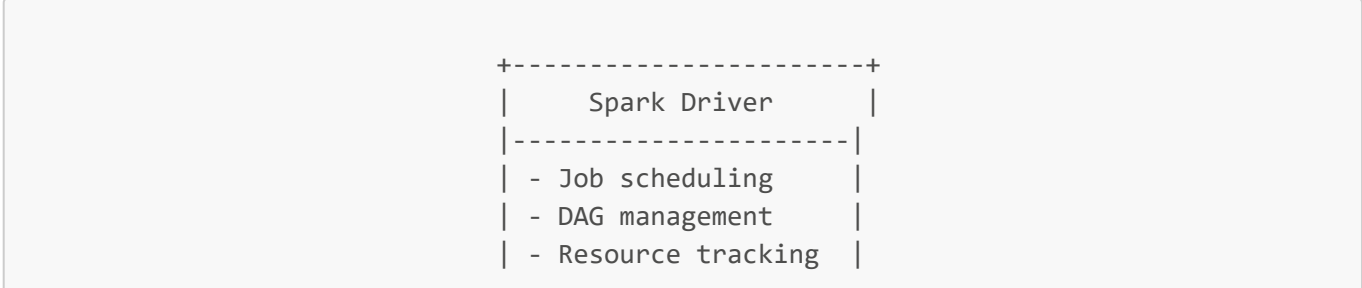
Running the logs producer (load generator). This should generate the data that the Spark application processes.

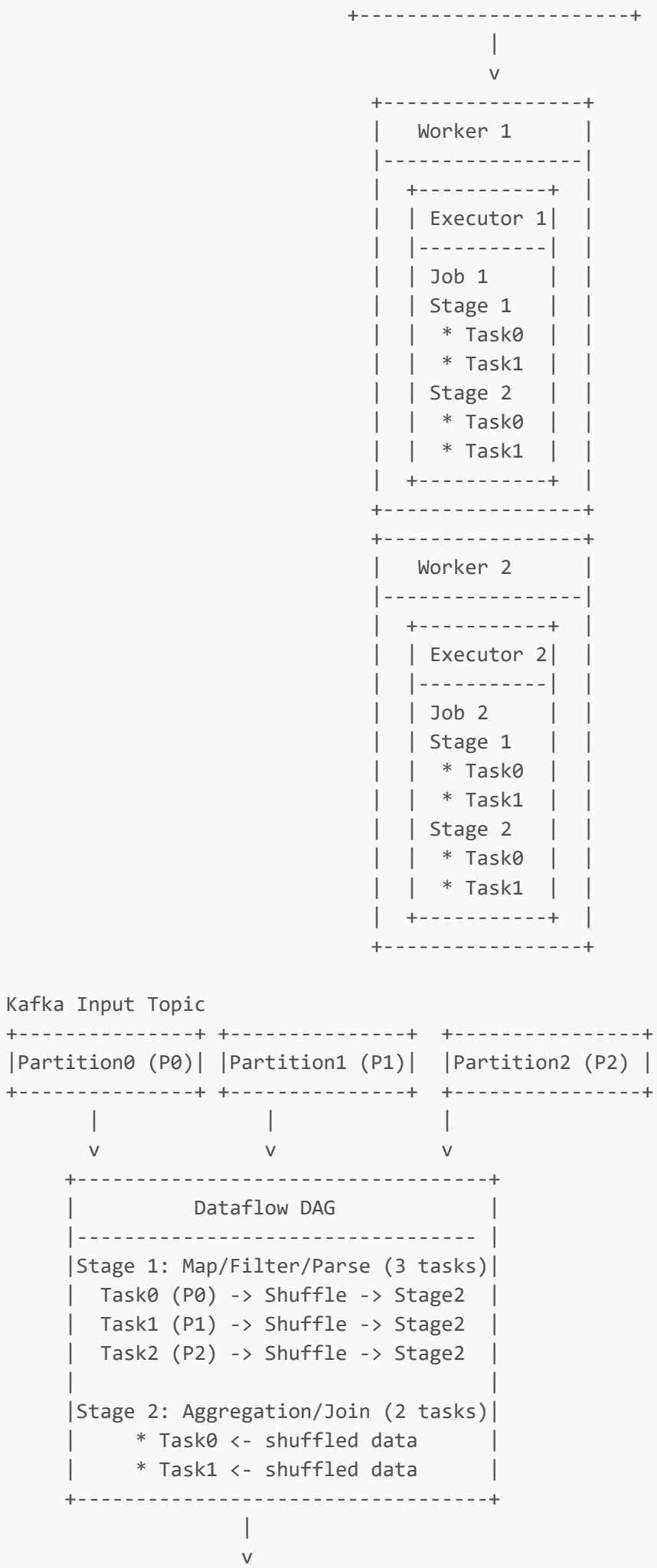
Inside the `load-generator` folder, revise the `docker-compose.yaml` file, especially the number of messages generated per second. To start the load generator:

```
docker compose up -d
```

Activity 1: Understanding the execution of Spark applications

Ilustration:

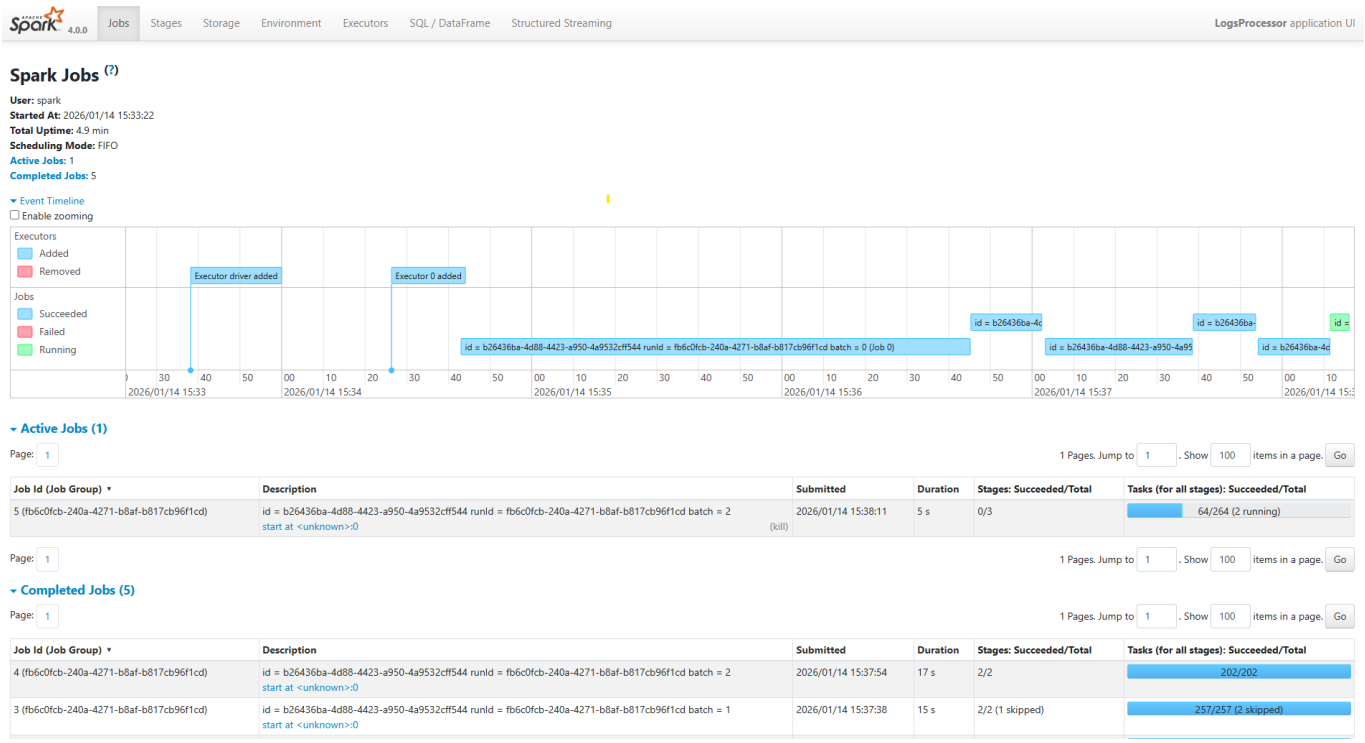



Worker 1
Executor 1
Job 1
Stage 1
* Task0
* Task1
Stage 2
* Task0
* Task1
Worker 2
Executor 2
Job 2
Stage 1
* Task0
* Task1
Stage 2
* Task0
* Task1
Kafka Input Topic
Partition0 (P0)
Partition1 (P1)
Partition2 (P2)
Dataflow DAG
Stage 1: Map/Filter/Parse (3 tasks)
Task0 (P0) -> Shuffle -> Stage2
Task1 (P1) -> Shuffle -> Stage2
Task2 (P2) -> Shuffle -> Stage2
Stage 2: Aggregation/Join (2 tasks)
* Task0 <- shuffled data
* Task1 <- shuffled data

```
                +-----------+
                |  Sink     |
                |(Kafka,    |
                | HDFS, etc)|
                +-----------+
```

1. Accessing the Interface

Once your Spark application is running, the Web UI is hosted by the **Driver**: http://localhost:4040



2. Key Concepts to Observe

As you navigate the UI, find and analyze the following sections to see Spark theory in action:

A. The Jobs Tab & DAG Visualization

Every **Action** (like `.count()`, `.collect()`, or `.save()`) triggers a Spark Job.

- **Task:** Click on a Job ID to see the **DAG Visualization**.
- **Concept:** Observe how Spark groups operations. Transformations like `map` or `filter` stay in one stage, while `sort` or `groupBy` create new stages.

This is the DAG-vis for the Job 0:

▼ DAG Visualization

Observation:

Stage 0 (Skipped) contains narrow transformations that are executed sequentially without shuffling data:

- **MicroBatchScan** reads records from Kafka partitions
- **Filter** removes unwanted records based on conditions
- **Project** selects specific columns from the data
- **WholeStageCodegen (1)** optimizes these operations into a single compiled code block
- **Exchange** operation triggers the shuffle, moving data between nodes

Stage 1 contains wide transformations that require data redistribution and state management:

- **Exchange** receives shuffled data from Stage 0
- **WholeStageCodegen (2)** and subsequent codegen blocks handle aggregation operations
- **StateStoreRestore** and **StateStoreSave** manage stateful streaming operations (indicating a `groupBy` or windowing operation that maintains state across micro-batches)
- **WholeStageCodegen (3) and (4)** continue processing with state information
- Final **Exchange** outputs the aggregated results

This visualization confirms the theory: narrow transformations (filter, map, project) remain in Stage 0, while the `groupBy` operation creates Stage 1 with shuffles and state management, demonstrating how Spark separates narrow and wide transformations into different stages.

B. The Stages Tab

Stages represent a set of tasks that can be performed in parallel without moving data between nodes.

- **Concept:** Look for **Shuffle Read** and **Shuffle Write**. This represents data moving across the network—the most "expensive" part of distributed computing.

shuffle:

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
135	id = b26436ba-4d88-4423-a950-4a9532cff544 runId = fb6c0fcb-240a-4271-b8af-b817cb96f1cd batch = 27 start at <unknown>:0 +details	2026/01/14 15:50:22	2 s	2/2				12.6 KiB
134	id = b26436ba-4d88-4423-a950-4a9532cff544 runId = fb6c0fcb-240a-4271-b8af-b817cb96f1cd batch = 26 start at <unknown>:0 +details	2026/01/14 15:50:21	0.3 s	80/80			7.9 KiB	
133	id = b26436ba-4d88-4423-a950-4a9532cff544 runId = fb6c0fcb-240a-4271-b8af-b817cb96f1cd batch = 26 start at <unknown>:0 +details	2026/01/14 15:50:07	14 s	200/200			12.6 KiB	7.9 KiB
131	id = b26436ba-4d88-4423-a950-4a9532cff544 runId = fb6c0fcb-240a-4271-b8af-b817cb96f1cd batch = 26 start at <unknown>:0 +details	2026/01/14 15:49:54	13 s	200/200			12.6 KiB	
130	id = b26436ba-4d88-4423-a950-4a9532cff544 runId = fb6c0fcb-240a-4271-b8af-b817cb96f1cd batch = 26 start at <unknown>:0 +details	2026/01/14 15:49:52	2 s	2/2				12.6 KiB
129	id = b26436ba-4d88-4423-a950-4a9532cff544 runId = fb6c0fcb-240a-4271-b8af-b817cb96f1cd batch = 25 start at <unknown>:0 +details	2026/01/14 15:49:52	0.3 s	71/71			7.9 KiB	
128	id = b26436ba-4d88-4423-a950-4a9532cff544 runId = fb6c0fcb-240a-4271-b8af-b817cb96f1cd batch = 25 start at <unknown>:0 +details	2026/01/14 15:49:38	13 s	200/200			12.7 KiB	7.9 KiB
126	id = b26436ba-4d88-4423-a950-4a9532cff544 runId = fb6c0fcb-240a-4271-b8af-b817cb96f1cd batch = 25 start at <unknown>:0 +details	2026/01/14 15:49:25	13 s	200/200			12.7 KiB	
125	id = b26436ba-4d88-4423-a950-4a9532cff544 runId = fb6c0fcb-240a-4271-b8af-b817cb96f1cd batch = 25 start at <unknown>:0 +details	2026/01/14 15:49:24	1 s	2/2				12.7 KiB
124	id = b26436ba-4d88-4423-a950-4a9532cff544 runId = fb6c0fcb-240a-4271-b8af-b817cb96f1cd batch = 24 start at <unknown>:0 +details	2026/01/14 15:49:23	0.2 s	79/79			7.9 KiB	
123	id = b26436ba-4d88-4423-a950-4a9532cff544 runId = fb6c0fcb-240a-4271-b8af-b817cb96f1cd batch = 24 start at <unknown>:0 +details	2026/01/14 15:49:11	13 s	200/200			12.7 KiB	7.9 KiB
121	id = b26436ba-4d88-4423-a950-4a9532cff544 runId = fb6c0fcb-240a-4271-b8af-b817cb96f1cd batch = 24 start at <unknown>:0 +details	2026/01/14 15:48:57	13 s	200/200			12.7 KiB	
120	id = b26436ba-4d88-4423-a950-4a9532cff544 runId = fb6c0fcb-240a-4271-b8af-b817cb96f1cd batch = 24 start at <unknown>:0 +details	2026/01/14 15:48:56	1 s	2/2				12.7 KiB
119	id = b26436ba-4d88-4423-a950-4a9532cff544 runId = fb6c0fcb-240a-4271-b8af-b817cb96f1cd batch = 23 start at <unknown>:0 +details	2026/01/14 15:48:55	0.2 s	77/77			7.9 KiB	

Highest shuffle read: 12.7 KiB Highest shuffle write: 12.7 KiB

C. The Executors Tab

This shows the "Workers" doing the actual computation.

- **Concept:** Check for **Data Skew**. If one executor has 10GB of Shuffle Read while others have 10MB, your data is not partitioned evenly.

only 2 Executors: 1 worker and the driver: so it is only partitioned on the one worker (3.2 MIB shuffle read)

Executors

Show 20 entries Search:

Executor ID	Address	Status	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Logs	Thread Dump	Heap Histogram	Add Time	Remove Time
driver	f0778eff64ec:37817	Active	0	35.6 MiB / 434.4 MiB	0.0 B	0	0	0	0	0	1.8 h (2 s)	0.0 B	0.0 B	0.0 B		Thread Dump	Heap Histogram	2026-01-14 18:02:34	-
0	172.21.0.5:35359	Active	0	35.6 MiB / 413.9 MiB	0.0 B	1	2	0	48911	48913	1.8 h (34 s)	0.0 B	3.2 MiB	2 MiB	stdout stderr	Thread Dump	Heap Histogram	2026-01-14 18:02:39	-

Showing 1 to 2 of 2 entries Previous 1 Next

---

3. Practical Exploration Questions

While your application is running, try to answer these questions:

1. **The Bottleneck:** Which Stage has the longest "Duration"? What are the technical reasons for it?

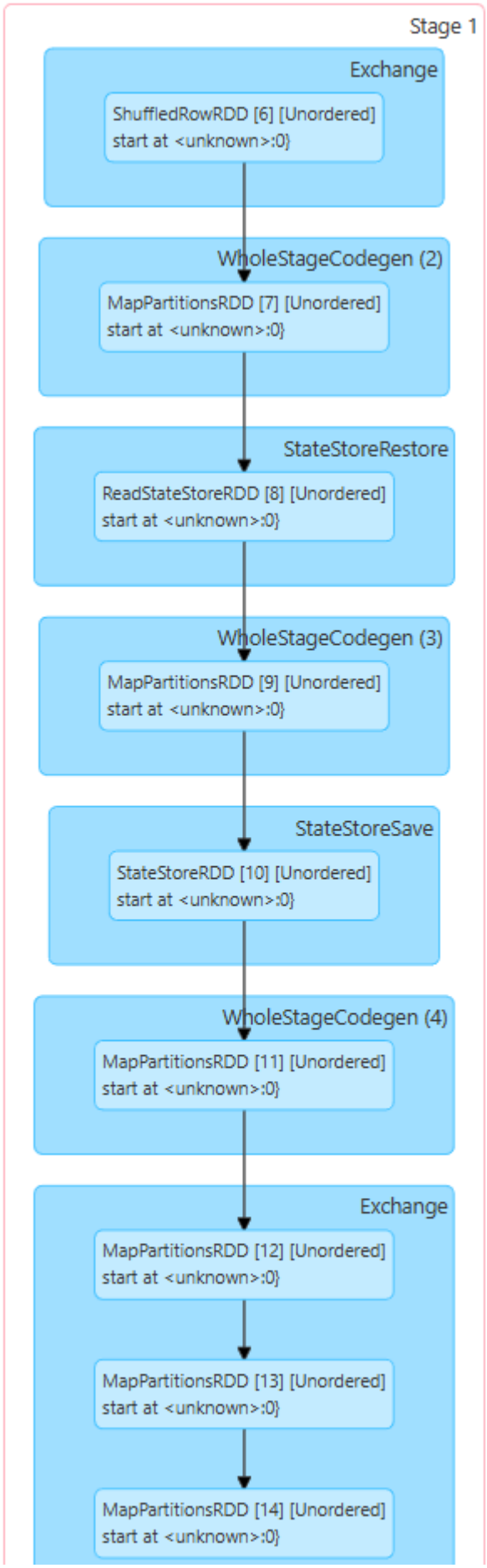
The Stage ID 1 -> 2 min

▾ Completed Stages (139)

Page: 1 2 >

2 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
1	id = b26436ba-4d88-4423-a950-4a9532cff544 runId = fb6c0fcb-240a-4271-b8af-b817cb96f1cd batch = 0 start at <unknown>:0 +details	2026/01/14 15:34:44	2.0 min	200/200				
173	id = b26436ba-4d88-4423-a950-4a9532cff544 runId = fb6c0fcb-240a-4271-b8af-b817cb96f1cd batch = 34 start at <unknown>:0 +details	2026/01/14 15:53:58	29 s	200/200			12.6 KiB	7.9 KiB



Root Causes:

1. **Multiple Shuffle Operations:**
   - **ShuffledRowRDD [6]** - Initial data redistribution from Stage 0
   - Multiple **MapPartitionsRDD** operations ([7], [9], [11], [12], [13], [14])
   - Final **Exchange** - Output shuffle
   - Each shuffle involves network I/O and serialization overhead
2. **State Management Chain:**
   - **StateStoreRestore** - Reads previous state from disk (RDD [8])
   - **StateStoreSave** - Writes updated state to disk (RDD [10])
   - Disk I/O is significantly slower than in-memory operations
   - State must be maintained for **streaming aggregations** (groupBy with watermarking)
3. **Complex Aggregation Pipeline:**
   - 4 separate **WholeStageCodegen** blocks (2, 3, 4, and implicit)
   - Multiple MapPartitions operations indicate complex transformations
   - Each codegen block represents a pipeline break requiring materialization
4. **Resource Constraints:**
   - **1 executor, 1 core, 1GB memory** forces **sequential execution** of 200 tasks
   - Cannot leverage parallelism across multiple cores
   - All 200 tasks queue and execute one-by-one

Comparison to Stage 0:

- Stage 0 only has narrow transformations (Filter, Project) - no shuffles, no state
- Stage 1 has wide transformations (groupBy) - requires shuffles AND stateful processing

2. **Resource Usage:** In the Executors tab, how much memory is currently being used versus the total capacity?

Storage Memory: 26.9 MiB / 848.3 MiB

Executors

Show Additional Metrics

Summary

	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Excluded
Active(2)	0	26.9 MiB / 848.3 MiB	0.0 B	1	2	0	18109	18111	44 min (13 s)	0.0 B	1.2 MiB	773 KiB	0
Dead(0)	0	0.0 B / 0.0 B	0.0 B	0	0	0	0	0	0.0 ms (0.0 ms)	0.0 B	0.0 B	0.0 B	0
Total(2)	0	26.9 MiB / 848.3 MiB	0.0 B	1	2	0	18109	18111	44 min (13 s)	0.0 B	1.2 MiB	773 KiB	0

Executors

Show 20 entries Search:

Executor ID	Address	Status	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Logs	Thread Dump	Heap Histogram	Add Time	Remove Time
driver	f0778eff64ec41631	Active	0	13.5 MiB / 434.4 MiB	0.0 B	0	0	0	0	0	23 min (2 s)	0.0 B	0.0 B	0.0 B		Thread Dump	Heap Histogram	2026-01-14 16:33:38	-
0	172.21.0.5:45593	Active	0	13.5 MiB / 413.9 MiB	0.0 B	1	2	0	18109	18111	21 min (11 s)	0.0 B	1.2 MiB	773 KiB	stdout stderr	Thread Dump	Heap Histogram	2026-01-14 16:34:26	-

Showing 1 to 2 of 2 entries Previous 1 Next

3. Explain with your own words the main concepts related to performance and scalability in the scenario of Spark Structured Streaming.

Answer:

Key Performance and Scalability Concepts:

1. **Micro-Batch Processing:**
    - Spark processes continuous streams in small batches (intervals)
    - Performance depends on processing batches faster than data arrives
    - If processing is slower than input rate → backpressure and delays
2. **Parallelism & Resources:**
    - **Executors** = workers processing data in parallel
    - **Cores** = simultaneous tasks per executor
    - **Memory** = handles shuffles and state without disk spillage
    - Current baseline (1 executor, 1 core, 1GB) = severe bottleneck
    - More resources = more parallel processing = higher throughput
3. **Shuffle Operations:**
    - Wide transformations (`groupBy`, `join`) redistribute data across nodes
    - Shuffle involves network I/O and serialization - most expensive operation
    - `spark.sql.shuffle.partitions` must match available parallelism
    - Data skew (uneven distribution) causes performance issues
4. **State Management:**
    - Stateful operations maintain data across micro-batches
    - `StateStoreRestore/Save` requires disk I/O for fault tolerance
    - State size grows with unique keys → impacts memory and performance
5. **Scalability:**
    - Add more executors across machines for horizontal scaling
    - Kafka partitions should match Spark parallelism
    - Spark UI helps identify bottlenecks (shuffle size, stage duration, data skew)

Conclusion: To achieve high throughput, increase executors, cores, and memory while tuning shuffle partitions to match hardware capacity.

Activity 2: Tuning for High Throughput

The Challenge

Your goal is to scale your application to process **several hundred thousand events per second are processed with batch sizes under 20 seconds to maintain reasonable event latency and data freshness**. On a standard laptop (8 cores / 16 threads), it is possible to process **1 million records per second** with micro-batch latencies staying below 12 seconds.

Please note that the `TARGET_RPS=10000` configuration in the docker compose file of the load generator. This value represents how many records per second each instance of the load generator should produce. The load generator can also run in parallel with multiple docker instances to increase the generation speed.

The Baseline Configuration

Review the starting configuration below. Identify which parameters are limiting the application's ability to use your hardware's full potential:

From the previous example of how to run the Spark application:

```
spark-submit \
  --master spark://spark-master:7077 \
  --packages org.apache.spark:spark-sql-kafka-0-10_2.13:4.0.0 \
  --num-executors 1 \
  --executor-cores 1 \
  --executor-memory 1G \
  /opt/spark-apps/spark_structured_streaming_logs_processing.py
```

Explenation:

- `--num-executors 1` limits the application to a single executor, preventing parallel processing across available CPU cores.
- `--executor-cores 1` restricts execution to one CPU core, severely underutilizing an 8-core / 16-thread machine.
- `--executor-memory 1G` provides insufficient memory for high-throughput streaming, limiting buffering, state handling, and shuffle performance.
- Overall, this configuration uses only a fraction of the available CPU and RAM, making it impossible to achieve high ingestion rates or low micro-batch latency.

Tuning Configurations (The "Knobs")

You must decide how to adjust the configurations to increase the performance. Consider the relationship between your **CPU threads**, **RAM availability**, and **Parallelism**. Examples of configurations

Parameter	Impact on Performance
`--num-executors`	Defines how many parallel instances (executors) run.
`--executor-cores`	Defines how many tasks can run in parallel on a single executor.
`--executor-memory`	Affects the ability to handle large micro-batches and shuffles in RAM.
`--conf "spark.sql.shuffle.partitions=2"`	Controls how many partitions are created during shuffles.

Tuned Configuration Strategy

To maximize throughput while keeping micro-batch latency under 20 seconds, the Spark configuration must be aligned with the available hardware resources:

- Increase the number of executors to enable parallel task execution
- Assign multiple CPU cores per executor to process Kafka partitions concurrently
- Increase executor memory to support large micro-batches and in-memory processing
- Reduce shuffle overhead by tuning the number of shuffle partitions
- Scale the load generator horizontally by running multiple instances (each producing `TARGET_RPS=10000`)

Example Tuned Spark Configuration

This configuration utilizes 8 CPU cores and sufficient memory, enabling high parallelism and allowing the application to process several hundred thousand events per second with micro-batch latencies below 20 seconds.

```
spark-submit \
  --master spark://spark-master:7077 \
  --packages org.apache.spark:spark-sql-kafka-0-10_2.13:4.0.0 \
  --num-executors 2 \
  --executor-cores 8 \
  --executor-memory 4G \
  --conf "spark.sql.shuffle.partitions=4" \
  /opt/spark-apps/spark_structured_streaming_logs_processing.py
```

I used the one from before with only 1 executer and 1 core - because i only have 1 cpu...

See full configuration: https://spark.apache.org/docs/latest/submitting-applications.html and general configurations: https://spark.apache.org/docs/latest/configuration.html. Also check possible configurations with:

```
spark-submit --help
```

Monitoring

Navigate to the **Structured Streaming Tab** in the UI to monitor the performance:

*** Input Rate vs. Process Rate:**

If your input rate is consistently higher than your process rate, your application is failing to keep up with the data stream.

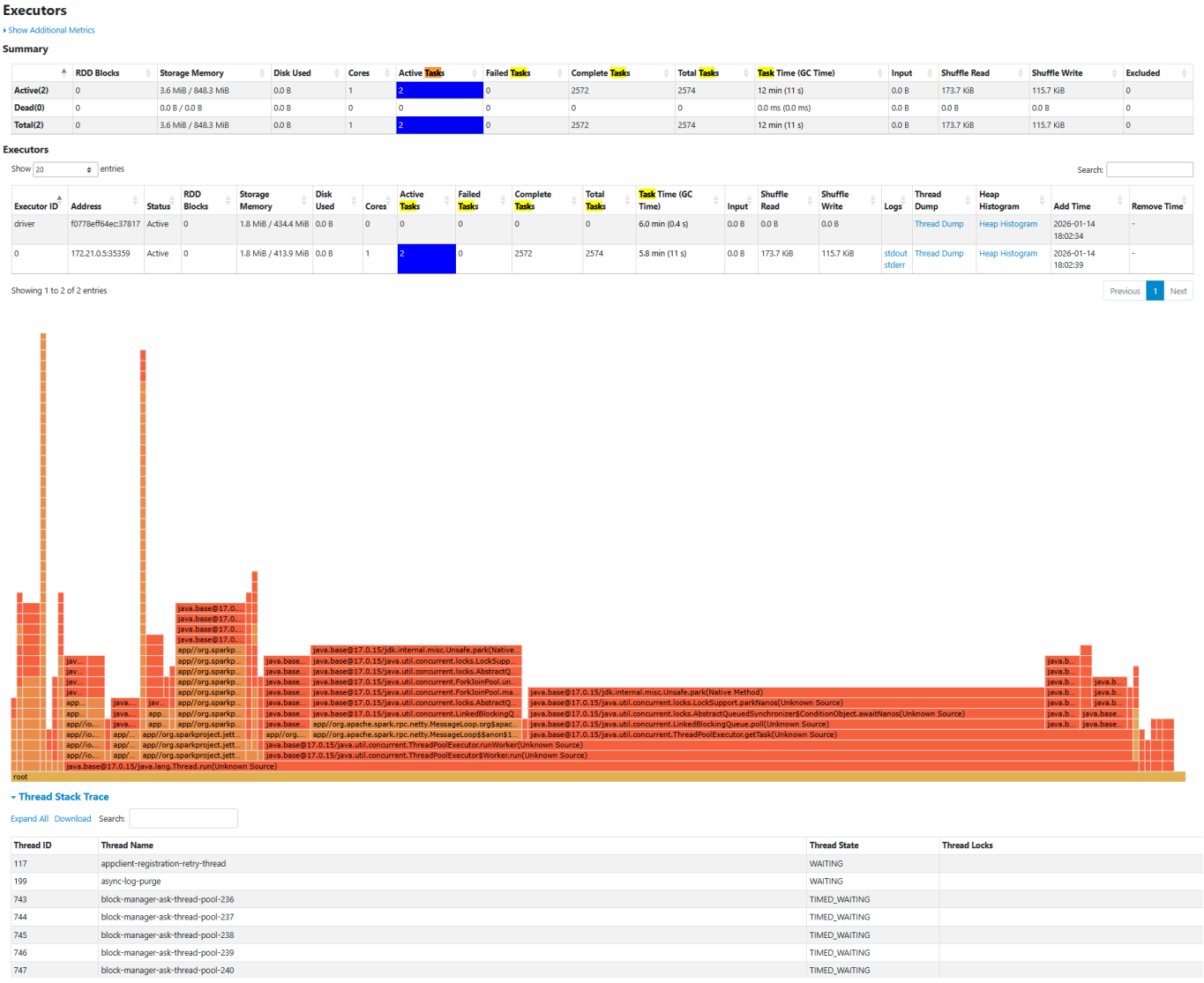
```
The input rate should closely match or stay below the process rate. If the input
rate is consistently higher, the application cannot keep up with the incoming
```

```
data, causing increasing micro-batch latency. This indicates insufficient CPU
resources or parallelism and requires increasing executors, cores, or Kafka
partitions.

Avg Input Rate / sec : 7495.93
Avg Process / sec    : 51179.87

so process is bigger than input -> it will keep up
```

The Executors Tab

In the The Executors Tab, check the **"Thread Dump"** and **"Task"** columns to verify resource utilization.

```
The Executors tab is used to verify effective resource utilization. The Task
column show multiple concurrent tasks running across executors. Thread dumps help
identify idle, blocked, or overloaded threads, indicating whether CPU resources
are underutilized or constrained by shuffle or I/O operations.
```



Executors

Show Additional Metrics

Summary

	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Excluded
Active(2)	0	3.6 MiB / 848.3 MiB	0.0 B	1	2	0	2572	2574	12 min (11 s)	0.0 B	173.7 KiB	115.7 KiB	0
Dead(0)	0	0.0 B / 0.0 B	0.0 B	0	0	0	0	0	0.0 ms (0.0 ms)	0.0 B	0.0 B	0.0 B	0
Total(2)	0	3.6 MiB / 848.3 MiB	0.0 B	1	2	0	2572	2574	12 min (11 s)	0.0 B	173.7 KiB	115.7 KiB	0

Executors

Show 20 entries Search:

Executor ID	Address	Status	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Logs	Thread Dump	Heap Histogram	Add Time	Remove Time
driver	f0778eff64ec37817	Active	0	1.8 MiB / 434.4 MiB	0.0 B	0	0	0	0	0	6.0 min (0.4 s)	0.0 B	0.0 B	0.0 B		Thread Dump	Heap Histogram	2026-01-14 18:02:34	-
0	172.21.0.5:35359	Active	0	1.8 MiB / 413.9 MiB	0.0 B	1	2	0	2572	2574	5.8 min (11 s)	0.0 B	173.7 KiB	115.7 KiB	stdout stderr	Thread Dump	Heap Histogram	2026-01-14 18:02:39	-

Showing 1 to 2 of 2 entries Previous 1 Next

Thread Stack Trace

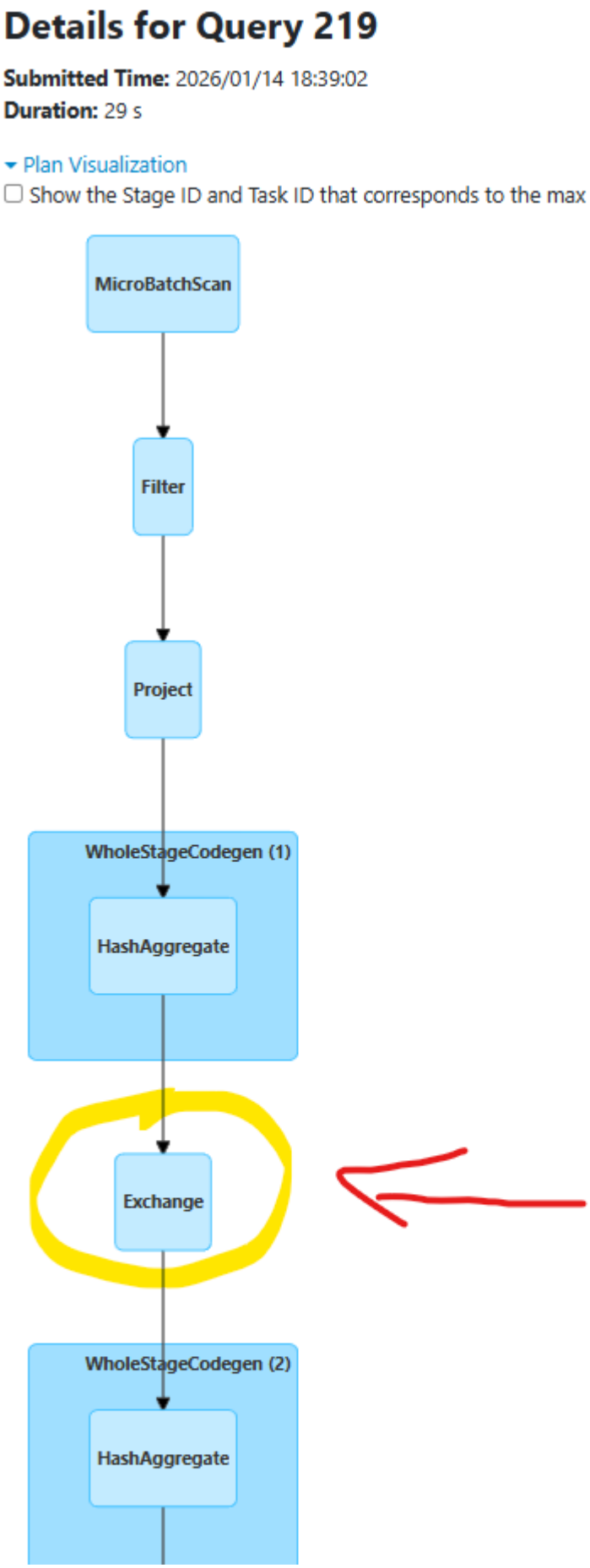
Expand All Download Search:

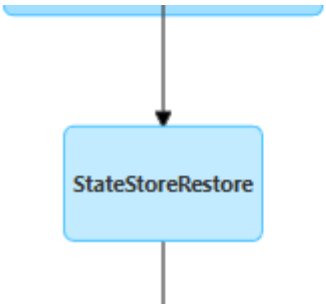
Thread ID	Thread Name	Thread State	Thread Locks
117	appclient-registration-retry-thread	WAITING	
199	async-log-purge	WAITING	
743	block-manager-ask-thread-pool-236	TIMED_WAITING	
744	block-manager-ask-thread-pool-237	TIMED_WAITING	
745	block-manager-ask-thread-pool-238	TIMED_WAITING	
746	block-manager-ask-thread-pool-239	TIMED_WAITING	
747	block-manager-ask-thread-pool-240	TIMED_WAITING	

The SQL/Queries Tab

Click on the active query to see the **DAG (Directed Acyclic Graph)**.

- **Identify "Shuffle" Boundaries:** Look for the exchange points where data is redistributed across the





cluster.

Exchange nodes in the DAG indicate shuffle operations, which are expensive and can limit throughput.

- **Identify Data Skew:** Is data being distributed evenly across all your cores, or are a few tasks doing all the work? Use the DAG to pinpoint which specific transformation is causing a bottleneck.
- Duration of the Querry: 30s

My DAG form the querry:

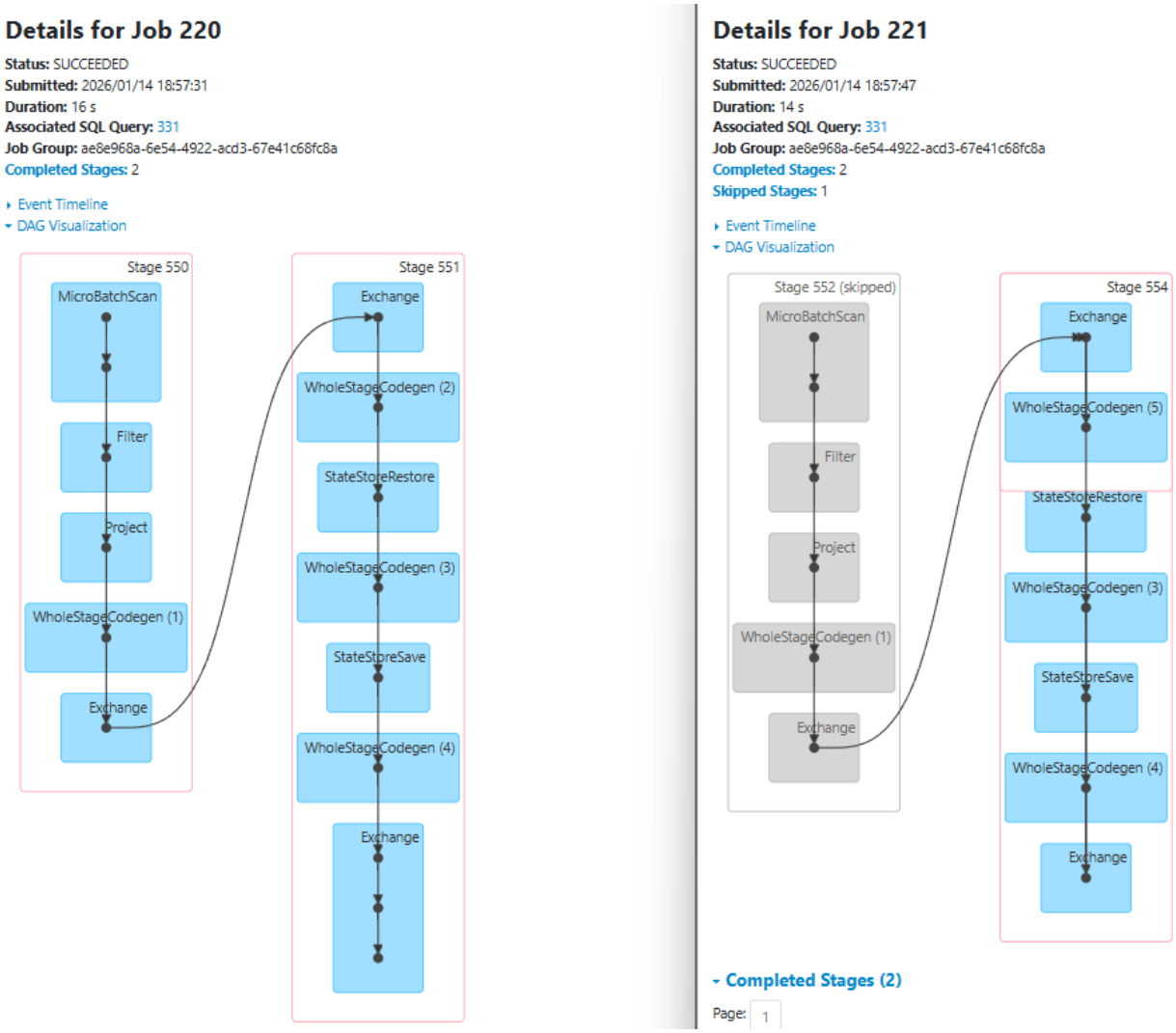
```
→ Filter
→ Project
→ HashAggregate
→ Exchange   ← first shuffle
→ HashAggregate
→ StateStoreRestore
→ HashAggregate
→ StateStoreSave
→ HashAggregate
→ Exchange   ← second shuffle
→ Sort
→ WriteToDataSourceV2
```

- Skew can only occur at shuffle points, i.e., the Exchange operations.
- The first Exchange feeds the next HashAggregate, and the second Exchange feeds Sort → Write.
- Everything before an Exchange is narrow transformation → no skew.
- Everything after an Exchange depends on how evenly the data was partitioned.

This Querry had 2 Jobs: Job 220 and Job 221

- Job 220 took 14s

- JOb 221 took 16s So they were evenly partitioned:



- **Submit activities 1 and 2 (answers and evidences) via Moodle until 20.01.2026**